# aula10

April 24, 2026

## 1 Linguagens e Ambientes de Programação

### 1.1 (Aula Teórica 10)

#### 1.1.1 Agenda

- Merge sort

[1]:
```
let rec split_n n l =
  if n = 0 then ([], l)
  else match l with
  | [] -> ([], l)
  | x::xs -> let (l1,l2) = split_n (n-1) xs in (x::l1,l2)

let split l =
  let n = List.length l in
  split_n (n/2) l
```

[1]:
```
val split_n : int -> 'a list -> 'a list * 'a list = <fun>
```

[1]:
```
val split : 'a list -> 'a list * 'a list = <fun>
```

[2]:
```
let _ = assert (split [1;2;3;4;5;6] = ([1;2;3], [4;5;6]))
let _ = assert (split_n 0 [1;2;3;4;5;6] = ([], [1;2;3;4;5;6]))
let _ = assert (split_n 1 [1;2;3;4;5;6] = ([1], [2;3;4;5;6]))
let _ = assert (split_n 10 [1;2;3;4;5;6] = ([1;2;3;4;5;6],[]))
```

[2]:
```
- : unit = ()
```

[2]:
```
- : unit = ()
```

[2]:
```
- : unit = ()
```

[2]:
```
- : unit = ()
```

[3]:
```
let rec merge l1 l2 =
  match l1, l2 with
  | [], l -> l
  | l, [] -> l
  | x::xs, y::ys -> if x < y then x::(merge xs l2) else y::(merge l1 ys)
```

[3]:
```
val merge : 'a list -> 'a list -> 'a list = <fun>
```

[4]:
```
let _ = assert (merge [1;3;5] [2;4;6] = [1;2;3;4;5;6])
let _ = assert (merge [1;3;5;7] [2;6] = [1;2;3;5;6;7])
let _ = assert (merge [] [1;3] = [1;3])
let _ = assert (merge [1;3] [] = [1;3])
```

[4]:
```
- : unit = ()
```

[4]:
```
- : unit = ()
```

[4]:
```
- : unit = ()
```

[4]:
```
- : unit = ()
```

[5]:
```
let rec mergesort l =
  match l with
  | [] | [_] -> l
  | _ -> let (l1,l2) = split l in merge (mergesort l1) (mergesort l2)
```

[5]:
```
val mergesort : 'a list -> 'a list = <fun>
```

[6]:
```
let _ = assert (mergesort [3; 2; 1; 4; 5; 6; 7; 8; 9; 10] = [1; 2; 3; 4; 5; 6;␣
 ↪7; 8; 9; 10])
```

[6]:
```
- : unit = ()
```

[7]:
```
let rec mergesort l =
  let rec split l = match l with
    | [] -> ([], [])
    | [x] -> ([x], [])
    | x::y::tl -> let (l1, l2) = split tl in (x::l1, y::l2)
```

```
  in let rec merge l1 l2 = match (l1, l2) with
    | ([], l) -> l
    | (l, []) -> l
    | (x::tl1, y::tl2) -> if x < y then x::(merge tl1 l2) else y::(merge l1 tl2)
  in match l with
  | [] | [_] -> l
  | _ -> let (l1,l2) = split l in merge (mergesort l1) (mergesort l2)
```

[7]:
```
val mergesort : 'a list -> 'a list = <fun>
```

[8]:
```
let _ = assert (mergesort [3; 2; 1; 4; 5; 6; 7; 8; 9; 10] = [1; 2; 3; 4; 5; 6;␣
 ↪7; 8; 9; 10])
```

[8]:
```
- : unit = ()
```

[9]:
```
let split_n n l =  (* tail recursive *)
  let rec split_n_rec n l acc =
    if n = 0 then (List.rev acc, l)
    else match l with
    | [] -> (List.rev acc, l)
    | x::xs -> split_n_rec (n-1) xs (x::acc)
  in split_n_rec n l []
```

[9]:
```
val split_n : int -> 'a list -> 'a list * 'a list = <fun>
```

[10]:
```
let _ = assert (split_n 0 [1;2;3;4;5;6] = ([], [1;2;3;4;5;6]))
let _ = assert (split_n 1 [1;2;3;4;5;6] = ([1], [2;3;4;5;6]))
let _ = assert (split_n 10 [1;2;3;4;5;6] = ([1;2;3;4;5;6],[]))
```

[10]:
```
- : unit = ()
```

[10]:
```
- : unit = ()
```

[10]:
```
- : unit = ()
```

[11]:
```
let merge l1 l2 = (* tail recursive *)
  let rec merge_rec l1 l2 acc =
    match l1,l2 with
    | [], l -> (List.rev acc)@l
    | l, [] -> (List.rev acc)@l
    | x::xs, y::ys -> if x < y then merge_rec xs l2 (x::acc) else merge_rec l1␣
 ↪ys (y::acc)
```

```
  in merge_rec l1 l2 []
```

[11]: val merge : 'a list -> 'a list -> 'a list = <fun>

[12]:
```
let _ = assert (merge [1;3;5] [2;4;6] = [1;2;3;4;5;6])
let _ = assert (merge [1;3;5;7] [2;6] = [1;2;3;5;6;7])
let _ = assert (merge [] [1;3] = [1;3])
let _ = assert (merge [1;3] [] = [1;3])
```

[12]: - : unit = ()

[12]: - : unit = ()

[12]: - : unit = ()

[12]: - : unit = ()